

CONTENTS

Purpose & Project Context	2
How the experiment was built	3
How the tests progressed from EDA to forecasting	5
Layer 1 -- Starting with Prompts Only	6
Layer 2 -- Using Rules for Recurring Constraints	7
Layer 3 -- Adding Claude.md	8
Layer 4 -- Adding Subagents	9
Layer 5 -- Wrapping Workflow in Commands	10
The Context Window and Usage	11
What the results showed overall	12
Findings that changed the Claude Code setup	13
Sources used	14

Purpose & Project Context

The purpose of this work was to explore different Agentic AI workflows and determine which Claude Code setup was most useful for strategic analysis tasks. Monitoring and forecasting Brent crude oil prices during the Iran-US conflict was used as the working use case because it required the agent to handle a realistic mix of data cleaning, statistical analysis, forecasting, file handling and interpretation.

The oil task was not only used to test whether Claude could produce charts or code. It provided a consistent environment where configuration mistakes became visible. For example, daily Brent prices and weekly conflict data could easily be mixed at the wrong grain; a model could look accurate while only learning that today's price is similar to yesterday's; and a technically complete run could still miss an important domain-specific check.

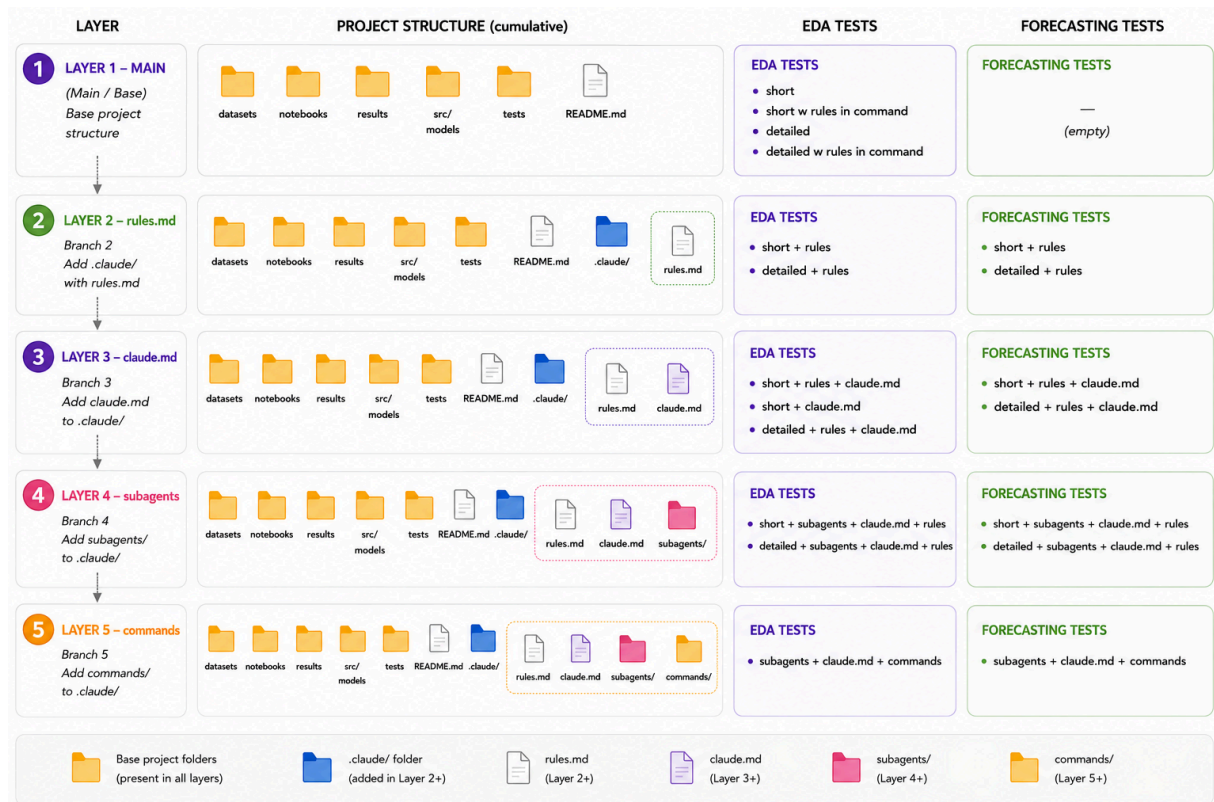
The workflows were compared mainly on five areas: output quality, setup effort, token/quota use, time taken and context-window use. The presentation also tracked lines of code, subagents spun up and clarifying questions because these helped explain why two runs with similar quality could behave very differently.

The presentation framed the configuration pieces as prompt styles, context window, subagents, rules, CLAUDE.md and architecture comparison. The same framing is retained here, but the components are described in the order they were introduced so the reasoning behind the final setup is easier to follow.

How the experiment was built

The project did not start with a complete agentic architecture. Testing began with the least configured version of the repository, observed what went wrong, and then added one layer at a time. Each layer inherited the previous layer.

The five cumulative layers were:



Layer	What was added	Why it was added
1 – Base project	Repository only; no CLAUDE.md, rules, subagents or commands	See what Claude does when almost everything must be inferred from the prompt and repository.
2 – Rules	.claude/rules/ plus safety/data/output rules	Move recurring constraints out of the prompt and prevent repeated file/data mistakes.
3 – CLAUDE.md	Project brief loaded as standing context	Give Claude the project objective, file locations, data grain, required checks and output expectations.

4 – Subagents	.claude/agents/ with focused workers	Move heavy exploration into separate contexts and make checks more repeatable.
5 – Commands	.claude/commands/ with reusable workflow command	Standardise how the multi-agent workflow is invoked.

The layers were built as a series of controlled configuration tests rather than simply building towards the most complex setup. At each layer, a different Claude Code feature was introduced and similar test cases were run again to observe whether that addition changed the result.

Each layer was also kept on a separate Git branch so that the configurations would not interfere with one another. Since files such as [CLAUDE.md](#), [.claude/rules/](#), subagent definitions and commands can automatically affect how Claude behaves, keeping everything in one working branch would make it difficult to know which instructions were active during a particular test. Separate branches created a fixed environment for each layer: [main](#) contained only the base project, Layer 2 added rules, Layer 3 added [CLAUDE.md](#), Layer 4 added subagents, and Layer 5 added commands.

Each layer therefore answered a different question. The base layer established how Claude performed when most decisions had to be inferred from the repository and prompt. Rules tested whether recurring instructions worked better as persistent files instead of repeatedly being included in the prompt. [CLAUDE.md](#) tested the value of providing project context automatically across sessions. Subagents tested whether specialised delegation could improve task coverage while moving heavier exploration into separate context windows. Commands then tested whether the final sequence could be packaged into a consistent, reusable invocation. The aim was not simply to find the configuration with the highest score, but to identify **which additions produced a useful improvement and which mainly added complexity or cost.**

How the tests progressed from EDA to forecasting

EDA was used to explore the widest set of configurations. Testing started with the completely raw setup, then compared short and detailed prompts, moved recurring instructions into rules, added a project brief, added subagents and finally wrapped the workflow in a command.

EDA	Layer	Prompt/configuration
E1	1	Short prompt – no persistent configuration
E2	1	Short prompt with rules written directly in the command
E3	1	Detailed prompt – no persistent configuration
E4	1	Detailed prompt with rules written directly in the command
E5	2	Short prompt + rules files
E6	2	Detailed prompt + rules files
E7	3	Short prompt + rules + CLAUDE.md
E8	3	Short/brief variant used to probe how much standing context could compensate for task detail
E9	3	Detailed prompt + rules + CLAUDE.md
E10	4	Short prompt + rules + CLAUDE.md + subagents
E11	4	Detailed prompt + rules + CLAUDE.md + subagents
E12	5	Rules + CLAUDE.md + subagents + command

By the forecasting stage, rules had already been established as part of the minimum setup. Re-running a configuration without them would deliberately reintroduce a file-safety problem that was not desirable in normal work. Forecasting therefore started from Layer 2 and used seven configurations rather than twelve.

Forecast	Layer	Configuration
F1	2	Short + rules
F2	2	Detailed + rules
F3	3	Short + rules + CLAUDE.md
F4	3	Detailed + rules + CLAUDE.md
F5	4	Short + rules + CLAUDE.md + subagents
F6	4	Detailed + rules + CLAUDE.md + subagents
F7	5	Rules + CLAUDE.md + subagents + command

Layer 1 -- Starting with Prompts Only

The first component tested was prompt style. The short prompt was intentionally close to a normal quick request: a single-line instruction that asked Claude to analyse the oil data without specifying the files, target variable, method or expected output. This meant Claude had to search the repository, decide which data grain was appropriate, infer what “driving the price” meant, and decide what form the result should take.

The detailed prompt asked for the same broad task but explicitly separated four pieces: DATA, TASK, FIND and OUTPUT. This became the clearest reusable prompt structure and was retained as the preferred way to specify analysis tasks.

Prompt	What it Specified	Why it helps
DATA	Exact files, relevant columns and time/grain information	Reduces repository exploration and prevents Claude from choosing the wrong dataset.
TASK	The precise analysis/test/model to run	Stops the agent from deciding the method from scratch.
FIND	The questions the result must answer	Makes the analysis interpretive rather than only producing code or charts.
OUTPUT	File type, format and destination	Makes the result reviewable and keeps outputs in the expected folders.

The raw runs exposed several useful failure modes. One early run merged weekly conflict information into daily rows and then treated same-day correlation as if it were predictive. Another fitted models on price levels and produced a very high R^2 , but the model had mostly learned persistence in the price series rather than useful structure in returns. Outputs could also arrive as scripts and image folders when a notebook or report would have been more reviewable.

The largest overall quality improvement in the experiment came from prompt specificity. Across comparable configurations, moving from a short prompt to a detailed one increased the quality score far more than adding another architecture feature.

Layer 2 -- Using Rules for Recurring Constraints

During the raw-prompt phase, two runs were observed where the repository README was deleted even though the task did not require it. The exact cause could not be reproduced afterwards, so the later absence of deletion is not treated as proof that the rules caused the fix. The observation instead showed that file safety should not depend on repeating the same instruction in every prompt.

Recurring constraints were moved into ``.claude/rules/``. The presentation showed two useful types: path-scoped data conventions and general file-safety rules. For example, conflict variables belonged with the weekly dataset, while price/sentiment belonged with daily data; outputs had fixed destinations; README.md, CLAUDE.md and existing result files were not to be overwritten.

This changed the role of the prompt. Instead of using prompt space for instructions like “never overwrite README” or “do not merge daily and weekly data,” the prompt could focus on the actual task. The rule files also applied to future prompts and subagents, which made them a better place for invariants than chat instructions.

The rules layer improved short-prompt performance, but it did not make short prompts equivalent to detailed prompts. Rules can constrain behaviour; they do not specify the exact analysis required for the current task.

Layer 3 -- Adding Claude.md

After the rules were stable, a root `CLAUDE.md` was added as a standing project context rather than as a giant instruction dump. Grouped the content into four practical areas: the brief, where things live, what to check, and how to finish.

CLAUDE.md	What was kept there
The brief	Project objective, target variable and sample window – facts Claude should not have to re-derive every run.
Where things live	Raw vs cleaned data, notebooks, reports and important project paths.
What to check	Pre-modelling checks and the instruction to report bad data rather than silently work around it.
How to finish	Expected deliverable location/format and what evidence each reported result should carry.

The CLAUDE.md evolved from actual failures. When outputs landed in the wrong place, the output contract became more explicit. When statistical checks were too vague, general wording such as “check the data” was replaced with specific checks. This is why the file was treated as a maintained project brief that absorbs stable lessons from repeated work rather than something designed perfectly upfront. The file was also kept concise, roughly under 200 lines, because it is standing context. Large reference material is better kept in separate files and read only when needed. The goal is to make the project legible every session without filling the context window with material that is irrelevant to most tasks.

A major result here was that CLAUDE.md could not rescue an under specified task. The short-prompt + brief configuration improved over a raw short prompt, but it still performed much worse than a detailed task specification. The short + CLAUDE.md run also asked the most clarifying questions and used a surprisingly large amount of context due to the need to guess what the prompt meant.

Layer 4 -- Adding Subagents

Subagents were added after the basic prompt, rules and project context were already working. Their purpose was to split long exploratory work into isolated contexts and make repeated checks more explicit.

The main thread then worked from short returned summaries instead of reopening every report produced by every worker. The expensive file reads and exploration can happen in the subagent's own context, while the main session only receives the distilled result.

The six run in sequence, each owning one stage so a failure lands where it belongs.

- **data-prep** owns the boundary between raw files and anything an analysis touches: profiling, the grain report, the eight-day publication anchor, and effective sample size; it exists separately because every leakage bug in this project entered during a merge, not during modelling.
- **eda-explorer** takes the prepared data and ranks candidate drivers: ADF and KPSS on every series, cross-correlation with a stated sign convention, VIF, the regime split, and a ranked table where every row carries direction, magnitude, lag and a p-value.
- **feature-engineer** builds the model frame: positive lags only, no centred windows, scaling deliberately left to the model pipeline so it refits per fold and reports the longest lookback, which is what the purge gap downstream has to cover.
- **forecast-builder** owns the harness: chronological expanding origins, benchmarks fitted *before* the roster, tuning on blocked CV, and a Diebold-Mariano test rather than an RMSE eyeball.
- **feature-importance** asks whether the importance ranking is stable across refits, since a ranking computed once describes the sample rather than the process.
- **backtest-auditor** deliberately **Read, Grep, Glob** with no write access, because an agent that can both find a leak and patch it can also quietly patch a leak and report a pass.

The agent definitions were also kept narrow. Each definition had a clear role, numbered steps, scope boundaries, allowed tools and an output/return contract.

The description was important because Claude can use it as a trigger for automatic delegation.

Layer 5 -- Wrapping Workflow in Commands

The final layer added a project-scoped slash command that invoked the analysis workflow. The command made the workflow easy to reuse and hid a long multi-step instruction behind a short entry point. This is useful when the same process will be run by different people or repeated many times.

However, the command did not make the underlying work cheaper. The command and the equivalent inline subagent configuration reached the same quality in both stages, while the command used substantially more tokens and quota in the observed runs. The presentation summarised this well: a one-line command hides the prompt, not the cost.

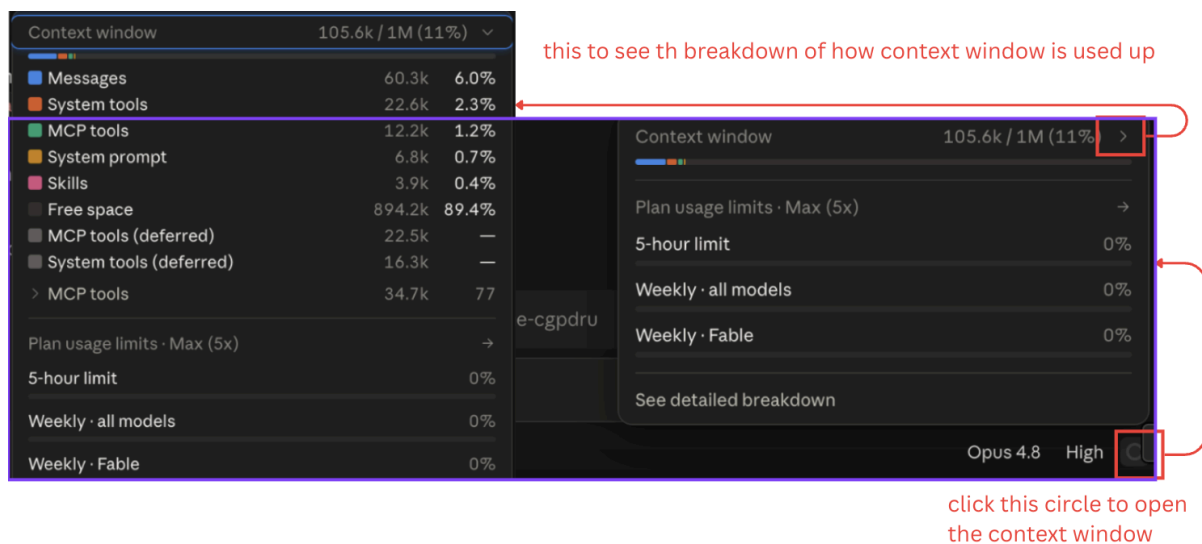
For that reason, a command is not necessary simply because it looks cleaner. It is more useful when standardisation matters: for example, when the team needs one repeatable `/run-analysis` entry point that consistently invokes the same workflow.

The Context Window and Usage

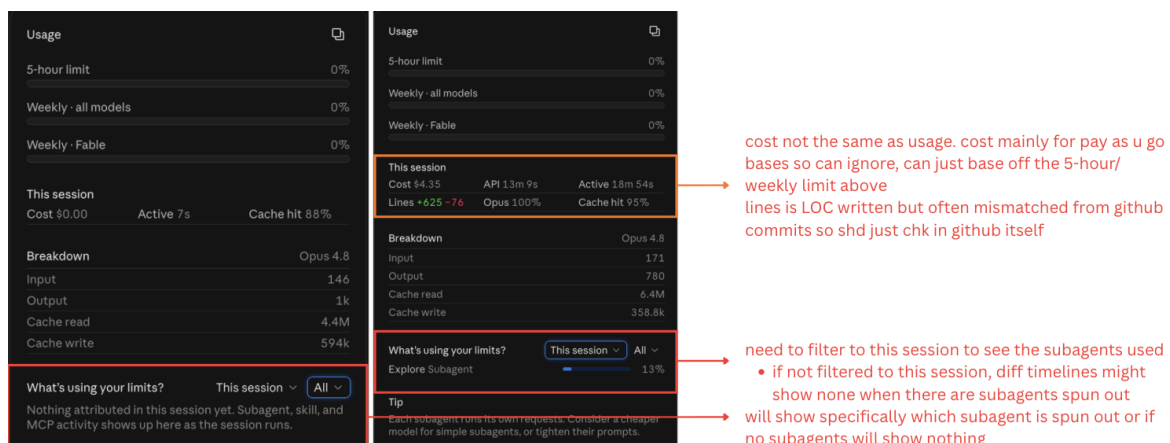
The context window became important because the runs were long. Claude's working context includes much more than user messages: system instructions, CLAUDE.md, tools, prompts, Claude responses, files read and terminal/tool outputs all consume the same working memory.

When the context becomes full, Claude Code can compact earlier history.

Compaction is useful, but early conversational detail or standing constraints typed only in chat may no longer remain verbatim. Persistent configuration such as rules and CLAUDE.md is therefore useful for information that must remain available across a long session.



*** diff versions hv diff context window recently some days will jump to 1million but its still in staged deployment. By default it shd be 200k



What the results showed overall

The most useful conclusion was not “use the most agentic configuration.” The results showed a hierarchy: first remove ambiguity from the task, then persist stable project knowledge, then add delegation only where the task is long enough to benefit from it.

Comparison	What was found
Short → detailed prompt	Write a proper task specification before adding more architecture.
Short → detailed with brief + subagents	Even a strong architecture cannot replace task detail.
Add project brief to short prompt	Standing project context helps, but only partially.
Add rules to short prompt	Rules remove known failure modes and reduce repeated instructions.
Brief, no subagents → brief + subagents	Delegation trades more total work for a cleaner main context.
Inline subagents → command	Use commands for standardisation/reuse, not because they are cheaper.
Plan vs accept-edits	Use plan mode when a wrong assumption is expensive;

In forecasting, the detailed accept-edits run achieved 100% of the test-case proxy with 4.37M tokens and 83.2k context tokens, while the command run also achieved 100% but used 12.98M tokens and 176.2k context tokens.

The detailed + subagent configuration was useful when the most complete analysis was required while protecting the main context. The detailed prompt without custom subagents was often the better efficiency choice when the task was already clear. The final recommendation is therefore conditional rather than one “best” architecture for everything.

Findings that changed the Claude Code setup

1. Short prompts can waste context, not save it.

A short prompt contains fewer user tokens, but it can force Claude to spend much more of the session searching files, inferring the task, asking questions and deciding what to do. In the forecasting comparison, the short + CLAUDE.md run took less time and produced less than half the code of the subagent run, yet consumed more main context. The extra context was being spent on exploration rather than output.

2. Persistent configuration for things that should not depend on memory.

File safety, data-grain conventions, project objectives and required checks are poor candidates for instructions that must be remembered and repeated in every prompt. Rules and CLAUDE.md make those constraints part of the environment instead of part of the operator's memory.

3. High rubric coverage does not guarantee analytical correctness.

All configurations missed an important issue around ACLED forward-filling/publication timing/effective sample size. This was the strongest warning from the study: configuration can make Claude more consistent at performing checks already known to be important, but it does not automatically invent every missing domain check. Once a new trap is discovered, it should be encoded into the rules, brief, rubric or a specialist agent. Subagents and commands also often produced longer and more complete outputs, but they also increased total work.

Sources used

The workflow configuration was developed from a combination of official Claude Code documentation, practitioner guides and open-source examples.

[15.1 Official Claude Code documentation – Create custom subagents](#)

This was the main source used to confirm how Claude Code subagents work at the product level. The documentation describes subagents as specialised assistants that run in their own context window, can be given a focused system prompt and restricted tool access, and return results to the main conversation. This directly supported the decision to use subagents as a context-management mechanism rather than treating them only as a way to parallelise work.

Heavy file reads and exploratory work could therefore happen inside the specialist context, while the main session received a shorter summary. This was particularly relevant to the observed context-window results, where a shorter prompt could still consume substantial main-session context because more exploration had to be performed centrally.

The documentation was also used to confirm the structure of project-level agent files under `.claude/agents/`, including the importance of the name and description fields and the ability to restrict tools.

[15.2 shanraishan – Claude Code Best Practice: Subagents](#)

This reference was useful as a compact implementation-oriented summary of the available subagent configuration fields. In particular, it reinforced the relationship between the description, tool allowlist, model, permission mode, skills, memory and other frontmatter settings. This helped frame the agent file as an executable configuration rather than simply a long prompt stored in Markdown.

The main idea carried into the project was to make invocation conditions explicit and keep capabilities intentional. For the oil-analysis agents, the role and trigger needed to be clear enough that a data-validation task would not be confused with relationship testing or forecasting work. This supported the use of narrow agents with distinct responsibilities instead of one large general analysis agent.

[15.3 vijaythecoder – Awesome Claude Agents: Best Practices](#)

This source contributed more directly to how the specialist agent files were structured. Its recommended pattern separates the routing description from the agent body, then structures the body as a small specification containing a mission or role, a repeatable workflow, an output contract and relevant checks. That pattern is close to the structure used in the project: role, numbered steps, scope and output.

The single-responsibility guidance was also relevant. The reference recommends avoiding large “mega-agents” and instead keeping one agent focused on one domain of expertise. This supported the decision to split the EDA workflow across separate specialists rather than placing data checks, stationarity, relationship testing and regime analysis into one very long subagent prompt. The goal was to make each agent easier to invoke, easier to inspect and less likely to carry irrelevant context.

[15.4 Product Talk – How to Use Claude Code](#)

This practitioner-oriented guide was used to understand how Claude Code features can be combined in day-to-day work. It was useful early in the project for separating the roles of prompts, persistent project instructions, agents and reusable commands, and for identifying which mechanisms were worth testing as separate layers.

The features were introduced progressively and compared. This eventually produced an important result: a command can make invocation more standardised and convenient without necessarily making the underlying execution cheaper. The command-based run reached similar quality to the direct subagent configuration but used more tokens and context, so commands were retained as a standardisation option rather than treated as an efficiency improvement.

[15.5 Robinson – Claude Code as a Data Analyst: From Zero to First Report](#)

This article was used as a worked comparison case rather than as a source of configuration guidance. It documents the same progression this project went through, starting from a prompt with no context and ending with a structured workflow, on an unrelated analytical task. The reported failures matched what was observed here in the raw-prompt runs: the agent guessed at table and column meanings and produced queries that either errored or returned the wrong quantity. Because each configuration in this study was run only once, an independent

account reaching the same conclusion was treated as useful corroboration of the finding that prompt specificity matters more than any other factor tested.

The article also contributed one idea that was not otherwise considered. Rather than relying on configuration files to carry instructions across a long session, it maintains a task list on disk that the agent reads, executes from, and updates each cycle. This keeps workflow state outside the conversation, so it survives compaction and allows an interrupted run to resume.

[15.6 VoltAgent - awesome-claude-code-subagents](#)

VoltAgent was used mainly as an example library for specialist-agent design. The value was in seeing how responsibilities can be decomposed into reusable specialist roles with their own instructions and outputs. This helped shape the decision to keep the project agents narrow and task-specific.

The examples also provided a useful contrast with the project's needs. Many public agent definitions are designed to be broadly reusable and can therefore be quite detailed. The oil-analysis workflow required a smaller, more controlled set of agents tied to known datasets, statistical checks and output locations. The external examples were therefore treated as design inspiration, while the final prompts were shortened and rewritten around the actual analysis tasks.

[15.7 luongnv89 - claude-howto: Subagents](#)

This reference was used alongside the official documentation when translating the subagent concept into actual project configuration files. It helped provide an implementation-level view of how subagents are defined and invoked, which supported the project's use of explicit roles, bounded scope and a defined return to the main session.

The main contribution to the project was the idea that an agent definition should make both its responsibility and its expected handoff clear. Full analytical work can be written to a report or result file, while the main session should receive only the concise information required for the next step. This was important because returning large intermediate outputs would remove much of the context-window benefit that subagents were intended to provide.

[15.8 Ist97 – claude-code-sub-agents](#)

This collection was reviewed as a second example library alongside VoltAgent, and was useful for a different reason. Where VoltAgent shows how a single specialist role is written, this repository shows how a set of specialists is organised and sequenced. The agents are grouped into domain folders, and a coordinating agent recommends which specialists should run in which phase of a larger task, producing a staged plan that the main session then executes. This is the same shape as the four-agent chain used in this project's analysis command, and reviewing it confirmed that an explicit ordering of narrow agents is a workable pattern rather than an improvisation. The difference is that the coordination here was fixed in advance in the command file, whereas the reference delegates the routing decision to an agent at run time. Fixing the order was used for a comparison study, since a routing agent would introduce variation between runs that could not be separated from the configuration being measured.

The collection also illustrated the cost of the approach it recommends. The repository's own worked example reports coordination across five or more agents in a multi-phase workflow consuming several hundred thousand tokens, which is consistent with this project's finding that delegation raises total token consumption while lowering peak context in the main session. Two further points limited its direct use. The agent definitions are broad full-stack engineering roles that assume a coordinating context service which does not exist in a plain Claude Code installation, so the same portions were removed here as were removed from the VoltAgent template. The repository also documents an installation problem in which non-agent markdown files placed alongside the agent definitions prevented the agents from loading, which reinforced the decision to keep this project's definitions in a dedicated directory containing nothing else.

Source	How it was used	Best use when continuing
Official Claude Code docs	Main product-level reference for subagent behaviour, isolated context windows, <code>.claude/agents/</code> structure, routing descriptions and tool restrictions. Supported the use of subagents primarily for context management , with heavy exploration occurring outside the main session.	Check current Claude Code behaviour, supported fields and syntax before making configuration changes.
shanraishan best-practice guide	Practical reference for subagent frontmatter and configuration fields such as description, tools, model, permissions, skills and memory. Reinforced explicit invocation conditions and intentionally restricted capabilities.	Use when creating or editing an agent configuration, then verify implementation details against the official documentation.
Awesome Claude Agents best practices	Influenced the role → workflow → checks → output structure of specialist agents and reinforced single-responsibility design instead of large “mega-agents”.	Use when designing a new narrow specialist and defining its responsibility, workflow and expected output.
Product Talk	Helped separate the roles of prompts, persistent instructions, agents and reusable commands when deciding which Claude Code mechanisms should be tested as separate configuration layers.	Use to identify new Claude Code workflow patterns or feature combinations worth testing.
Robinson – Claude Code as a Data Analyst	Used as an independent comparison case for the raw-prompt findings. Similar failures from missing context supported the observation that prompt specificity was a major factor. It also introduced the idea of storing	Use when extending the workflow beyond configuration files, particularly for persistent task tracking and

	task state on disk so work can survive compaction or interruption.	resumable long-running analysis.
VoltAgent examples	Used as an example library for specialist decomposition and reusable agent roles. The patterns were adapted into a smaller, project-specific set of analytical agents rather than copied directly.	Browse for design ideas when a new specialist role is justified, then simplify and adapt it to the project's datasets and checks.